

Baseline ML Model

```
In [1]: import pickle
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

from sklearn.linear_model import LinearRegression, ElasticNet, LassoCV, RidgeCV
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from lightgbm import LGBMRegressor

import optuna

In [2]: with open('../data/processed/dataset.pkl', 'rb') as f:
dataset = pickle.load(f)

In [3]: dataset.keys()

Out[3]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

In [4]: X_train = dataset['X_train']
X_test = dataset['X_test']
X_val = dataset['X_val']
y_train = dataset['y_train']
y_test = dataset['y_test']
y_val = dataset['y_val']

In [5]: lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_val)
print("Linear Regression")
print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("R2 Score:", r2_score(y_val, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

Linear Regression
Mean Squared Error: 0.7238494476388447
R2 Score: 0.2747745696494136
Mean Absolute Error: 0.6608437771781829

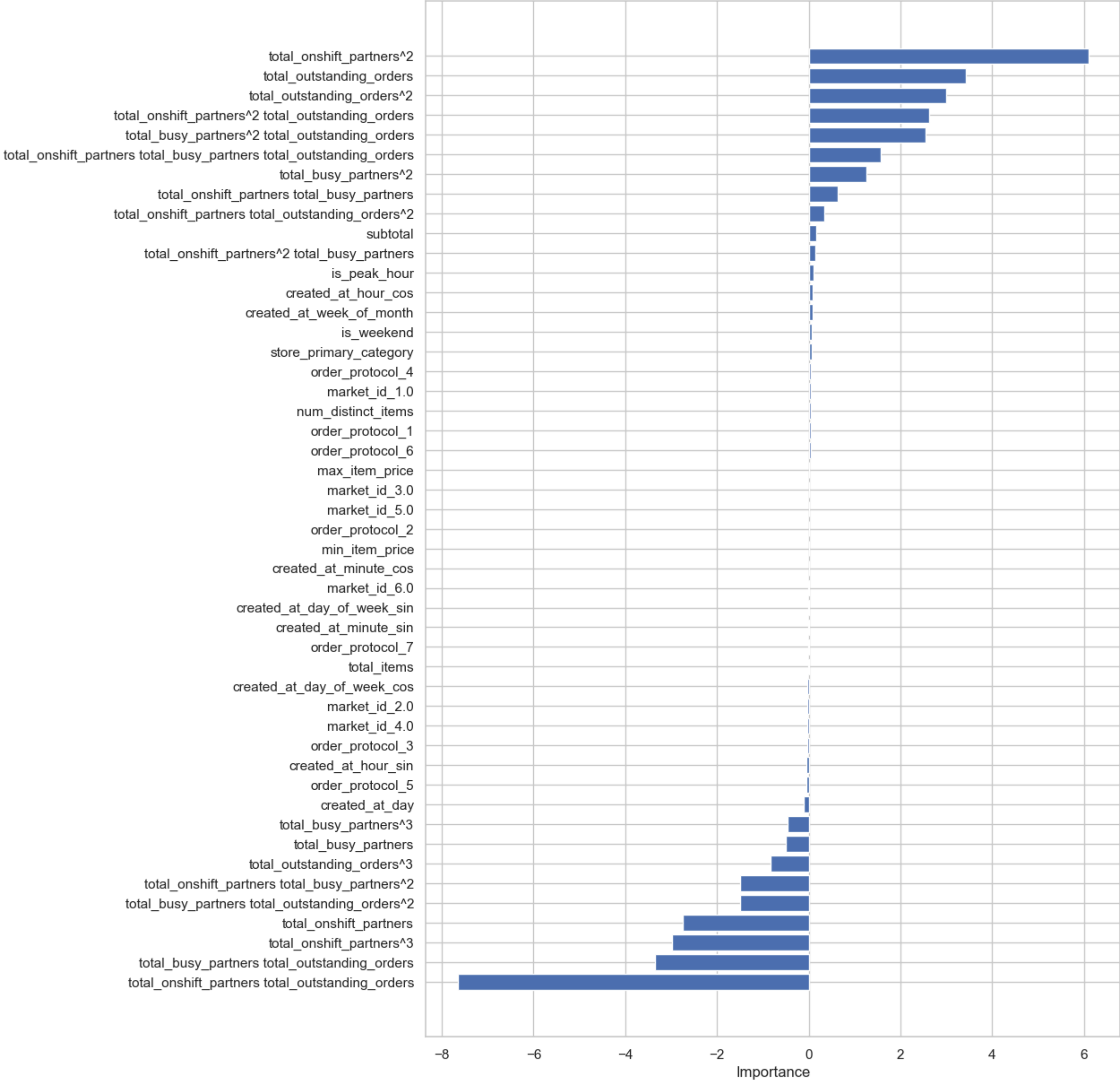
In [6]: C=[0.1, 1, 10]

for c in C:
    lasso = LassoCV(alphas=[c], cv=5)
    lasso.fit(X_train, y_train)
    y_pred = lasso.predict(X_val)
    print(f"Lasso Regression (alpha={c})")
    print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
    print("R2 Score:", r2_score(y_val, y_pred))
    print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))
    ridge = RidgeCV(alphas=C)
    ridge.fit(X_train, y_train)
    y_pred = ridge.predict(X_val)
    print("Ridge Regression")
    print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
    print("R2 Score:", r2_score(y_val, y_pred))
    print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

Lasso Regression (alpha=0.1)
Mean Squared Error: 0.9070163455659324
R2 Score: 0.0912594853891967
Mean Absolute Error: 0.7415297818944618
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167
Lasso Regression (alpha=1)
Mean Squared Error: 0.9981092966643812
R2 Score: -6.626476704241213e-06
Mean Absolute Error: 0.7850793418708032
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167
Lasso Regression (alpha=10)
Mean Squared Error: 0.9981092966643812
R2 Score: -6.626476704241213e-06
Mean Absolute Error: 0.7850793418708032
Ridge Regression
Mean Squared Error: 0.7238708808917491
R2 Score: 0.27475309565352657
Mean Absolute Error: 0.660850102072167

In [9]: importance = lr.coef_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 15))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()
```

Feature Importance

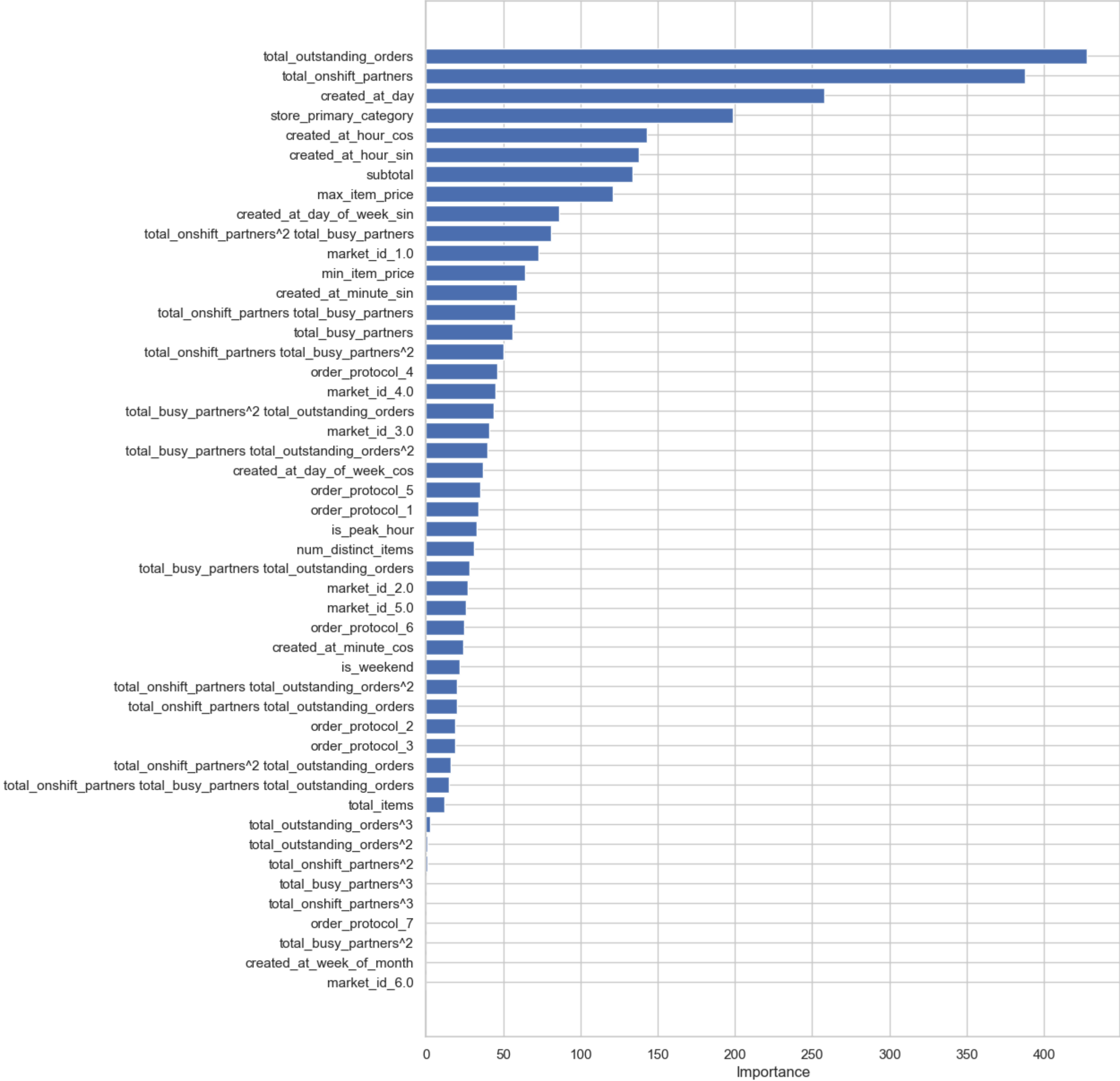


```
In [10]: lgbm = LGBMRegressor()
lgbm.fit(X_train, y_train)
y_pred = lgbm.predict(X_val)
print("LightGBM")
print("Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("R2 Score:", r2_score(y_val, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_val, y_pred))

[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.006536 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 5507
[LightGBM] [Info] Number of data points in the train set: 124099, number of used features: 47
[LightGBM] [Info] Start training from score -0.000000
LightGBM
Mean Squared Error: 0.6615931175671896
R2 Score: 0.33714924426654636
Mean Absolute Error: 0.6306848036268882
```

```
In [12]: importance = lgbm.feature_importances_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 15))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()
```

Feature Importance



```
In [ ]: # set up optuna study
study = optuna.create_study(direction="minimize")
def objective(trial):
    # hyperparameter tuning
    params = {
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'num_leaves': trial.suggest_int('num_leaves', 20, 100),
        'max_depth': trial.suggest_int('max_depth', 3, 12),
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),
        'min_child_samples': trial.suggest_int('min_child_samples', 1, 20)
    }
    lgbm = LGBMRegressor(**params)
    lgbm.fit(X_train, y_train)
    y_pred = lgbm.predict(X_val)
    mse = mean_squared_error(y_val, y_pred)
    return mse
# optimize hyperparameters
study.optimize(objective, n_trials=100)
# print best hyperparameters
best_params = study.best_params
print("Best Hyperparameters:", best_params)
# train final model with best hyperparameters
```

Best Hyperparameters: {'learning_rate': 0.07752425203437846, 'num_leaves': 97, 'max_depth': 11, 'n_estimators': 153, 'min_child_samples': 19}

```
In [11]: lgbm = LGBMRegressor(**best_params)
lgbm.fit(X_train, y_train)
y_pred = lgbm.predict(X_val)
print("Validation Mean Squared Error:", mean_squared_error(y_val, y_pred))
print("Validation R2 Score:", r2_score(y_val, y_pred))
importance = lgbm.feature_importances_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 10))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.002314 seconds. You can set `force\_row\_wise=true` to remove the overhead. And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 1823

[LightGBM] [Info] Number of data points in the train set: 124099, number of used features: 31

[LightGBM] [Info] Start training from score -0.000000

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

Validation Mean Squared Error: 0.6448919586547537

Validation R2 Score: 0.35388215081104113

Feature Importance

